

NAME: JASIM ASMAAN A

REG NO:192421060

COURSE: COMPUTER ARCHITECTURE

COURSE CODE: CSA1208

CO2-AT2

Q1. 8-bit 2's Complement Addition with Overflow Detection

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int a = 120; 6 int b = 50; 7 int result; 8 9 printf("A = %d\n", a); 10 printf("B = %d\n", b); 11 12 result = a + b; 13 14 printf("Result = %d\n", result); 15 16 // 8-bit signed range is -128 to +127 17 if (result > 127 result < -128) 18 { 19 printf("Overflow detected!\n"); 20 printf("The result cannot be represented in 8-bit signed form.\n"); 21 } 22 else 23 { 24 printf("No overflow.\n"); 25 } 26 27 return 0; 28 }</pre>		<pre>A = 120 B = 50 Result = 170 Overflow detected! The result cannot be represented in 8-bit signed form. === Code Execution Successful ===</pre>

Q2. Carry Look-Ahead Adder (CLA) – C Program

```

main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     char a[5], b[5];
6     int A[4], B[4];
7     int P[4], G[4];
8     int C[5];
9     int i;
10
11     printf("Enter 4-bit binary number A: ");
12     scanf("%4s", a);
13
14     printf("Enter 4-bit binary number B: ");
15     scanf("%4s", b);
16
17     /* Convert input characters to bits */
18     for (i = 0; i < 4; i++)
19     {
20         A[3 - i] = a[i] - '0';
21         B[3 - i] = b[i] - '0';
22     }
23
24     /* Generate and Propagate */
25     for (i = 0; i < 4; i++)
26     {
27         G[i] = A[i] & B[i];
28         P[i] = A[i] ^ B[i];
29     }
30
31     /* Initial carry */
32     C[0] = 0;
33
34

```

```

Enter 4-bit binary number A: 1011
Enter 4-bit binary number B: 0110

Generate signals: 0010
Propagate signals: 1101
Carry signals: 11100
Sum: 0001
Carry Out: 1

--- Code Execution Successful ---

```

Q3 – Booth's Multiplication

```

main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int M, Q;
6     int A = 0, Q_1 = 0;
7     int count = 4;
8     int Q0, i;
9
10    printf("Enter multiplicand: ");
11    scanf("%d", &M);
12
13    printf("Enter multiplier: ");
14    scanf("%d", &Q);
15
16    printf("Initial values:\n");
17    printf("M = %d\n", M);
18    printf("Q = %d\n", Q);
19    printf("A = %d\n", A);
20    printf("Q_1 = %d\n", Q_1);
21
22    while (count > 0)
23    {
24        Q0 = Q & 1;
25
26        printf("\tStep %d", 5 - count);
27        printf("\tQ0 Q_1 = %d%d", Q0, Q_1);
28
29        if (Q0 == 0 && Q_1 == 1)
30        {
31            A = A + M;
32            printf("\tA = A + M = %d", A);
33        }
34        else if (Q0 == 1 && Q_1 == 0)
35        {
36            A = A - M;
37            printf("\tA = A - M = %d", A);
38        }
39        else
40        {
41            printf("\tNo operation");
42        }
43
44        /* Save Q0 before shifting */
45        Q_1 = Q0;
46
47        /* Arithmetic right shift */
48        A = A >> 1;
49        Q = Q >> 1;
50
51        /* Sign extension for negative A */
52        if (A < 0)
53        {
54            Q = Q | (1 << 3);
55        }
56    }
57

```

```

Enter multiplicand: -6
Enter multiplier: 5

Initial values:
M = -6
Q = 5
A = 0
Q_1 = 0

Step 1:
Q0 Q_1 = 10
A = A - M = 6
After arithmetic right shift:
A = 3
Q = 2
Q0 Q_1 = 01
A = A + M = -3
After arithmetic right shift:
A = -2
Q = 9
Q0 Q_1 = 10
A = A - M = 4
After arithmetic right shift:
A = 2
Q = 4
Step 4:
Q0 Q_1 = 01
A = A - M = -4
After arithmetic right shift:
A = -2
Q = 10

Final Product = -22

--- Code Execution Successful ---

```

Q4 – Non-Restoring Division

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int dividend, divisor;
6     int A = 0, Q, M;
7     int count = 4;
8     int i;
9
10    printf("Enter dividend: ");
11    scanf("%d", &dividend);
12
13    printf("Enter divisor: ");
14    scanf("%d", &divisor);
15
16    Q = dividend;
17    M = divisor;
18
19    printf("\nNon-Restoring Division\n");
20
21    for (i = 0; i < count; i++)
22    {
23        // Left shift A and Q by 1
24        A = (A << 1) | ((Q >> 3) & 1);
25        Q = (Q << 1) & 15;
26
27        if (A >= 0)
28        {
29            A = A - M;
30        }
31        else
32        {
33            A = A + M;
34        }
35
36        if (A >= 0)
37        {
38            Q = Q | 1;
39        }
40
41        printf("\tStep %d", i + 1);
42        printf("\tA = %d", A);
43        printf("\tQ = %d\n", Q);
44    }
45
46    // Final correction
47    if (A < 0)
48    {
49        A = A + M;
50    }
51
52    printf("\tQuotient = %d", Q);
53    printf("\tRemainder = %d\n", A);
54
55    return 0;
56 }
```

Enter dividend: 13
Enter divisor: 3
Non-Restoring Division

Step 1
A = -2
Q = 10

Step 2
A = 0
Q = 5

Step 3
A = -3
Q = 10

Step 4
A = -2
Q = 4

Quotient = 4
Remainder = 1

=== Code Execution Successful ===

Q5 – IEEE 754 Floating-Point Addition

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     float large, small, result;
6     double result2;
7
8     large = 100000000.0f;
9     small = 0.0000001f;
10
11    result1 = large + small;
12    result2 = (double)large + (double)small;
13
14    printf("Large number = %.10f\n", large);
15    printf("Small number = %.10f\n", small);
16
17    printf("\nUsing single precision:\n");
18    printf("Result = %.10f\n", result1);
19
20    printf("\nUsing double precision:\n");
21    printf("Result = %.10f\n", result2);
22
23    printf("\nDebugging:\n");
24    printf("The small number may lose precision when added to the large number.\n");
25
26    printf("\nOptimization:\n");
27    printf("Use double precision for better accuracy.\n");
28
29    return 0;
30 }
```

Large number = 100000000.0000000000
Small number = 0.0000000001

Using single precision:
Result = 100000000.0000000000

Using double precision:
Result = 100000000.000000000377808

Debugging:
The small number may lose precision when added to the large number.

Optimization:
Use double precision for better accuracy.

=== Code Execution Successful ===